

Proyecto de persistencia con SQLite: clientes y vehiculos

Enunciado

Desarrolla una aplicacion siguiendo una arquitectura por capas.

La aplicacion debe gestionar **clientes** y **vehiculos** usando una base de datos **SQLite**.

Reglas del dominio

- La clave principal del cliente es su **dni**.
- Un cliente puede tener **uno o varios vehiculos**.
- Cada vehiculo pertenece a un unico cliente.
- Las validaciones principales deben hacerse en la capa **service**.
- El acceso a datos debe hacerse desde la capa **repository**.

Objetivos

Debes de ser capaz de:

- organizar un proyecto en paquetes **app**, **model**, **repository** y **service**
- crear clases de modelo con atributos, constructores, getters, setters, **equals**, **hashCode** y **toString**
- implementar CRUD sobre SQLite con **PreparedStatement**
- validar datos sencillos antes de persistirlos
- separar reglas de negocio y acceso a datos
- probar el comportamiento de la capa **service** con tests automatizados
- restaurar una base de datos de copia de seguridad antes de cada test

Modelo de datos

Tabla **cliente**

- **dni** TEXT PRIMARY KEY
- **nombre** TEXT NOT NULL
- **email** TEXT
- **telefono** TEXT
- **ciudad** TEXT
- **activo** INTEGER NOT NULL

Tabla **vehiculo**

- **id** INTEGER PRIMARY KEY AUTOINCREMENT
- **matricula** TEXT NOT NULL UNIQUE
- **marca** TEXT NOT NULL
- **modelo** TEXT NOT NULL

- **color** TEXT
 - **anio** INTEGER
 - **kilometros** INTEGER NOT NULL
 - **precio** REAL NOT NULL
 - **vendido** INTEGER NOT NULL
 - **dni_cliente** TEXT NOT NULL
-

Tareas minimas a implementar

Cliente

- crear cliente
- buscar cliente por DNI
- listar todos los clientes
- actualizar cliente
- eliminar cliente por DNI
- listar clientes activos
- buscar clientes por ciudad

Vehiculo

- crear vehiculo
 - buscar vehiculo por id
 - listar todos los vehiculos
 - actualizar vehiculo
 - eliminar vehiculo por id
 - listar vehiculos de un cliente
 - listar vehiculos vendidos
 - listar vehiculos disponibles
 - cambiar el propietario de un vehiculo
 - marcar un vehiculo como vendido
 - actualizar kilometros
-

Reglas de validacion sugeridas

ClienteService

- el DNI no puede ser nulo ni vacio
- el nombre no puede ser nulo ni vacio
- no se puede crear un cliente con un DNI repetido
- no se puede actualizar un cliente que no existe

VehiculoService

- la matricula no puede ser nula ni vacia
- marca y modelo son obligatorios
- **kilometros** no puede ser negativo

- **precio** debe ser mayor que 0
 - no se puede crear un vehiculo si el cliente no existe
 - no se puede cambiar el propietario a un DNI inexistente
 - no se puede vender dos veces el mismo vehiculo
-

Estructura del proyecto

```
src/main/java/com/ejemplo/  
├── app  
├── model  
├── repository  
│   └── sqlite  
└── service
```

```
src/test/java/com/ejemplo/  
├── service  
└── support
```

Paquetes incluidos en esta propuesta

model

- **Cliente**
- **Vehiculo**

repository

- **IClienteRepository**
- **IVehiculoRepository**

repository/sqlite

- **SQLiteConnectionManager**
- **SchemaRepository**
- **ClienteSqliteRepository**
- **VehiculoSqliteRepository**

service

- **ClienteService**
- **VehiculoService**

support en test

- **TestBackupManager**

- `TestDataFactory`
-

Base de datos de test

Los tests deben usar una base de datos de trabajo que se restaura desde una **copia de seguridad** antes de cada test.

La idea es:

1. partir siempre del mismo estado inicial
2. evitar que un test afecte a otro
3. poder repetir la bateria completa de tests con resultados estables

Base de datos.

La base de datos se encuentra en la siguientes rutas:

- `src/main/resources/data/sqlite/vehiculos.db;`
 - `src/main/resources/data/sqlite/vehiculos_backup.db;`
-

Pruebas automatizadas

La suite de tests verifica:

- creacion correcta de clientes y vehiculos
- rechazo de datos invalidos
- actualizacion y borrado
- filtros por activo, ciudad, vendido y propietario
- cambios de propietario
- marcado de vendido
- calculos sencillos en servicios
- aislamiento entre tests restaurando la BBDD en cada ejecucion

En esta propuesta se incluyen **mas de 40 tests** centrados en el comportamiento de los servicios.

Calificacion automatica

La nota se calcula de forma automatica a partir de:

- los reportes XML generados por `mvn test` en `target/surefire-reports`
- la documentacion del `README.md`

Reparto de pesos

- **Cliente = 4 puntos**
 - 3 puntos por tests del bloque `cliente`
 - 1 punto por documentacion de las operaciones de cliente en el `README.md`
- **Vehiculo = 6 puntos**

- 5 puntos por tests del bloque **vehiculo**
- 1 punto por documentacion de las operaciones de vehiculo en el **README.md**

Como se evalua la documentacion

El script **tools/calcular_nota.py** revisa automaticamente que en el **README.md** aparezcan las operaciones del enunciado dentro de las secciones:

- **### Cliente**
- **### Vehiculo**

Si faltan operaciones documentadas, la puntuacion de documentacion baja de forma proporcional.

Uso

1. Ejecutar los tests:

```
mvn test
```

2. Calcular la nota:

```
mvn clean verify -Pcalificar
```

3. Revisar el resultado en:

```
target/nota.txt
```

Ejecucion

Ejecutar tests

```
mvn test
```

Ejecutar aplicacion de ejemplo

```
EJECUTA EL MAIN PARA VERIFICAR EL FINDALL DE LOS SERVICIOS
```

IMPORTANTE: La conexión externa y no permitida supone la exclusión del examen y la suspensión del mismo